UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Agentic Legal RAG System

Architecture and Evaluation Report

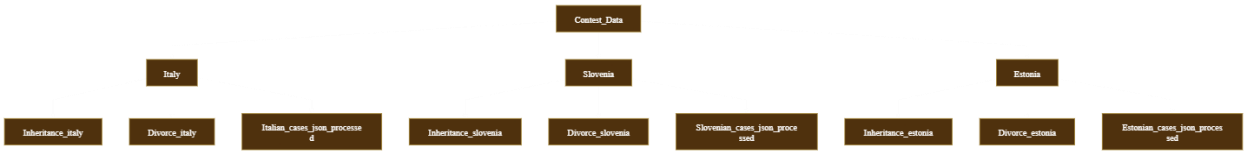
Submitted to:
Prof. Flora Amato

Submitted by:
Francesco Ventimiglia [D03000137]
Danilo Rodriguez [D03000194]
Hussain Ullah Baig [D03000205]

Text Mining Project — Academic Evaluation

1. System Overview

This project implements a Retrieval-Augmented Generation (RAG) pipeline designed to answer natural language queries about civil law — specifically inheritance and divorce proceedings — across three European jurisdictions: Italy, Estonia, and Slovenia. The system was built and evaluated as part of a Text Mining academic project, with the goal of comparing four progressively sophisticated agent architectures on both retrieval quality and generation quality.



At its core, the system accepts a user query in natural language, retrieving the most relevant legal documents from a pre-built vector store. All four architectures share the same underlying document corpus and vector store infrastructure, differing only in how they orchestrate retrieval, filtering, and answer generation.

Corpus Structure

The document corpus is composed of two broad categories of legal text:

- statutory legal codes (e.g., civil code articles governing property, succession, and marital dissolution)
- court decisions (procedural records, judgment summaries, and case outcomes). These were sourced for all three jurisdictions, providing both normative and jurisprudential coverage of the legal domains.

- Legal codes: statutory text and civil code articles governing inheritance and divorce
- Court decisions: case outcomes, procedural records, and judgment summaries
- Four FAISS indexes: divorce_cases, divorce_codes, inheritance_cases, inheritance_codes
- Metadata fields per document:
  - law (Divorce / Inheritance),
  - country (Italy / Estonia / Slovenia), doc_type (case / code)
- Evaluation dataset: 10 held-out question-answer pairs per domain, used exclusively for RAGAS scoring

2. Agent Architectures

All four architectures operate on top of a shared infrastructure layer. This ensures that any observed performance differences between architectures are attributable solely to differences in retrieval strategy and answer synthesis logic, rather than to variations in the underlying models, embedding space, or document chunking.

Documents are chunked using a RecursiveCharacterTextSplitter with:

- a window of 512 tokens and
- a 50-token overlap between consecutive chunks.

This overlap ensures that sentences spanning chunk boundaries are not arbitrarily truncated, preserving semantic continuity especially in longer legal provisions. Each chunk is embedded using the all-MiniLM-L6-v2 model (384 dimensions) and stored in a FAISS flat index alongside its metadata.

The vector stores support filtered retrieval based on the law and country metadata fields, allowing pre-filter narrowing before similarity search.

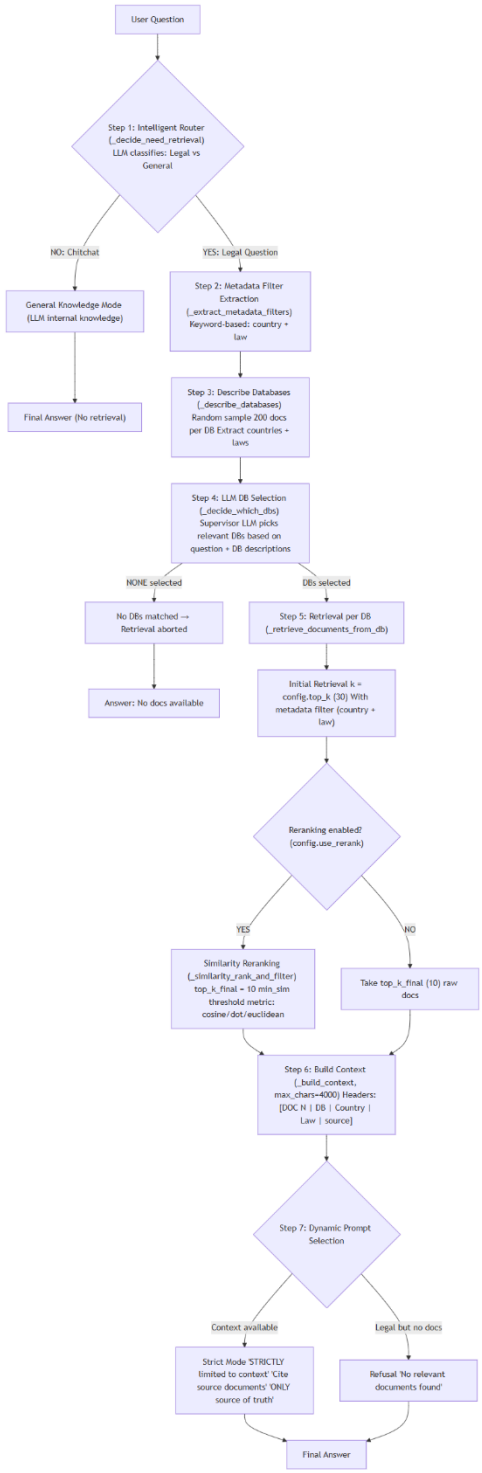
Component	Implementation
Embedding model	all-MiniLM-L6-v2 (384-dim); swappable to BAAI/bge-base-en-v1.5 (768-dim)
Chunking	512-token chunks, 50-token overlap (RecursiveCharacterTextSplitter)
Vector store	FAISS flat index; 4 separate indexes with metadata pre-filtering
LLM	GPT-4o-mini via OpenRouter; max_tokens=768, temperature=0.2
Cross-encoder	cross-encoder/ms-marco-MiniLM-L-6-v2 (logit threshold ≥ 0.0)
BM25 fusion	Reciprocal Rank Fusion (RRF, k=60) combining dense + sparse ranks

The cross-encoder and BM25+RRF components are only active in the Hybrid and Hybrid Multi-Agent architectures. The cross-encoder assigns a relevance logit to each (query, document) pair; documents falling below a threshold of 0.0 are discarded before generation.

The BM25 component captures exact lexical matches — particularly important for legal text where article numbers, Latin phrases, and statute identifiers must match precisely — and its rank is fused with the dense FAISS rank via Reciprocal Rank Fusion.

3. Types of Agents Implemented

The four architectures form a spectrum from minimal orchestration (Single Agent) to full hierarchical multi-agent synthesis with hybrid retrieval (Hybrid Multi-Agent). Each builds on the lessons of the previous one, isolating specific design choices and their effects on the five RAGAS metrics.



3.1 Single Agent

The Single Agent is the baseline configuration and represents the simplest possible end-to-end RAG pipeline. A single LLM instance serves as both the retrieval planner and the answer generator, with no routing, no sub-agent decomposition, and no fallback logic.

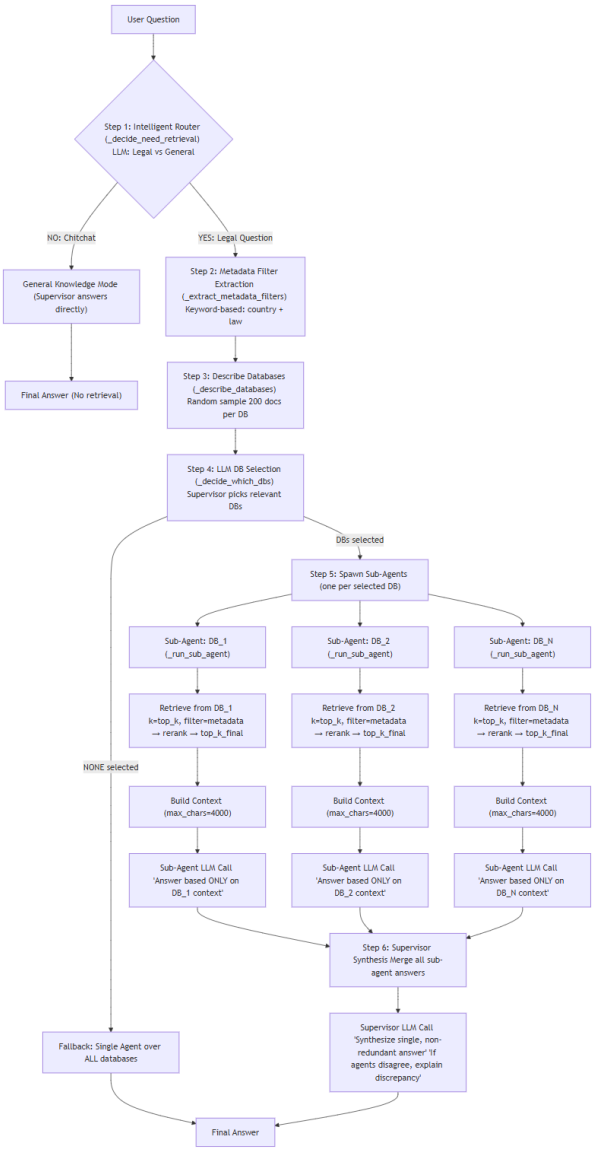
The pipeline begins with a keyword-based metadata filter that extracts the likely legal domain (Divorce or Inheritance) and jurisdiction (Italy, Estonia, or Slovenia) directly from the surface form of the query using pattern matching. These filters are applied as pre-conditions on the FAISS search, narrowing the candidate pool before vector similarity is computed. The top_k candidates are retrieved, optionally reranked by cosine similarity to produce a final top_k_final documents, and all ten are concatenated into a single context window for the LLM.

This architecture achieves the highest answer correctness score across all configurations, likely because the model receives the full, unpartitioned set of retrieved documents without any loss introduced by summarization or synthesis steps.

However, the answer relevancy can be hampered by limitation of large context windows, like when given ten heterogeneous legal documents, the model tends to produce broad, multi-topic answers rather than tightly focused responses to the specific question asked.

3.2 Multi-Agent

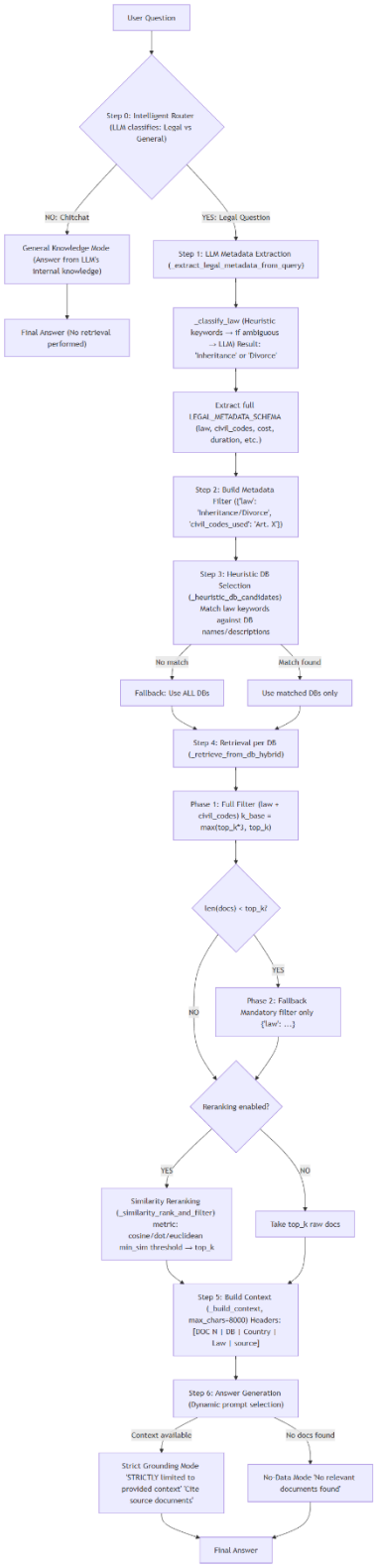
The Multi-Agent architecture introduces two additional LLM calls on top of the Single Agent: an intelligent router at the front and a supervisor synthesizer at the back. The router's role is to classify the incoming query as either a legal/specific question requiring document retrieval, or a general/chitchat query that can be answered directly from parametric knowledge. This prevents the system from performing unnecessary retrieval on off-topic or definitional questions, reducing hallucination risk for those cases.



When retrieval is needed, the router also performs LLM-based database routing, selecting which of the four FAISS indexes are relevant to the query. Unlike the keyword heuristic in the Single Agent, this routing step uses the LLM's reasoning ability to infer the legal domain from more ambiguous phrasings. Each selected database is then assigned its own sub-agent: an independent LLM call that retrieves documents from that database only and generates a partial answer within its isolated context window. Finally, the supervisor LLM receives all sub-agent answers and synthesizes them into a single coherent response, resolving any contradictions and citing the most relevant evidence.

The sub-agent isolation leads to a meaningful improvement in faithfulness because each sub-agent operates on a smaller, more topically consistent set of documents. Answer relevancy also improves significantly, since each sub-agent produces a focused answer to a narrower sub-question.

The trade-off is a drop in answer correctness, which occurs because the supervisor's synthesis process can sometimes dilute or partially drop specific factual details that were present in individual sub-agent answers but were deemed less salient during consolidation.



3.3 Hybrid RAG

Hybrid RAG architecture is motivated by the observation that keyword-based metadata extraction is brittle for legal queries, where domain terminology may not match simple surface patterns.

It replaces the heuristic filter with a structured LLM metadata extraction step, where the supervisor LLM parses the query into a JSON object conforming to the LEGAL_METADATA_SCHEMA. This schema mandates the law field (Divorce or Inheritance) and optionally extracts additional fields such as civil_codes_used, cost, duration, and nature_of_separation.

The retrieved candidates pass through a two-phase fallback strategy:

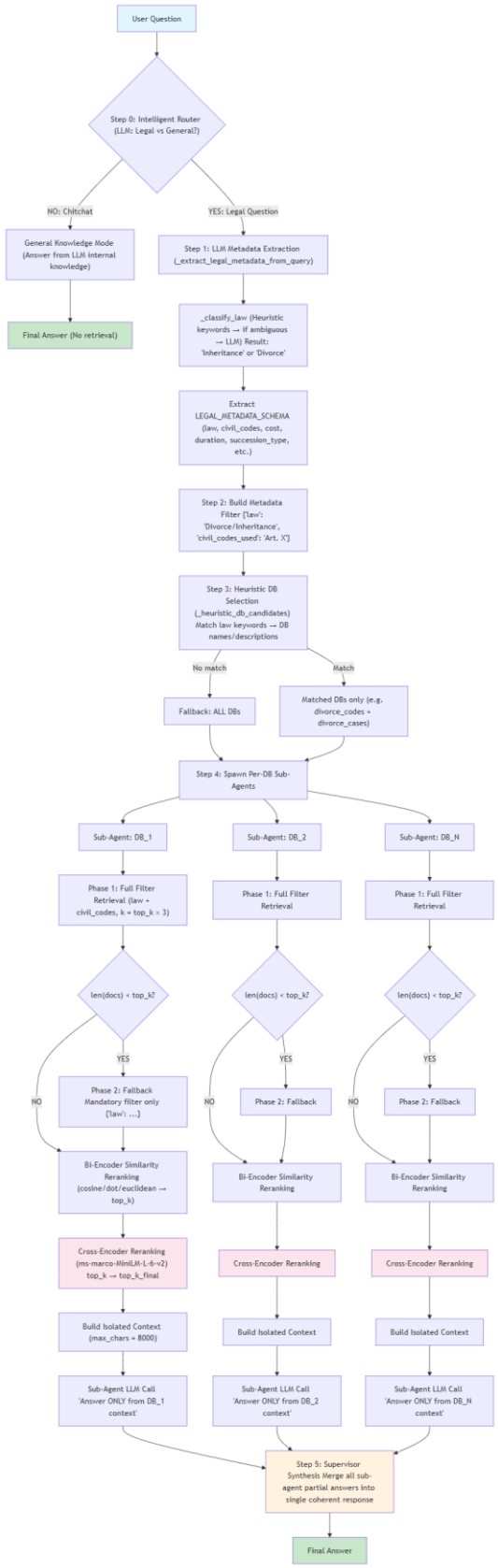
- Phase 1 applies all extracted metadata fields as FAISS pre-filters; if fewer than top_k documents are returned,
- Phase 2 retries with only the mandatory law field to avoid an empty context.

After dense retrieval, BM25 scores are computed over the same corpus, and the two ranked lists are fused via Reciprocal Rank Fusion. The resulting fused list is optionally reranked by cosine similarity, then filtered by the cross-encoder — documents with a logit below 0.0 are discarded.

The surviving documents, along with the extracted metadata string, are passed to a single LLM call for answer generation.

This pipeline produces the best retrieval metrics of all architectures: context precision reaches (almost every retrieved document is relevant). The structured metadata injected into the prompt also helps the LLM frame its answer within the correct legal and jurisdictional context.

However, the architecture's critical weakness is identical to the Single Agent's: all retrieved documents are fed to one LLM call in one pass. So, with higher top_k_final value, the context window is too large and heterogeneous, and the model's answer relevancy collapses to the lowest of all configurations.



3.4 Hybrid Multi-Agent

The Hybrid Multi-Agent architecture resolves the central weakness of plain Hybrid RAG by combining its superior retrieval pipeline with the per-DB sub-agent synthesis of the Multi-Agent architecture. LLM metadata extraction, heuristic DB routing, two-phase FAISS retrieval, BM25+RRF fusion, cosine reranking, and cross-encoder threshold filtering remain the same.

What changes is that, instead of feeding all retrieved documents to a single LLM call, the documents are partitioned back by their source database.

Each partition is assigned to a dedicated sub-agent LLM, which generates a partial answer from its isolated, topically coherent context window. A final supervisor LLM then synthesizes all sub-agent answers alongside the extracted metadata string and a cross-checked evidence summary into a single coherent response.

The effect is dramatic: the same large, retrieved pool that collapsed answer relevancy in plain Hybrid RAG is now distributed across sub-agents, each of which receives a compact and focused set of documents. Hence, the answer relevancy and faithfulness can improve.

The optimal hyperparameter configuration is top_k=15, top_k_final=10. This setting provides sufficient retrieval breadth for good context recall while keeping each sub-agent's context window small enough to maintain high faithfulness and relevancy.

Increasing to top_k=20 or top_k=30 degrades both faithfulness and relevancy monotonically, confirming that the performance optimum lies at the compact end of the context size range.

4. Evaluation Results (RAGAS)

All metrics scored on [0, 1]. The target threshold is 0.80. Bold values indicate the best per column. These metrics were obtained by comparing against the ground truth (GT) of each question that were kept hidden in the site [https://textmining_exam.idealunina.work] for ragas evaluations.

Architecture	Prec	Recall	Faith	Relev	Correct	Mean
Single Agent (30/10)	0.800	0.742	0.780	0.641	0.695 ★	0.732
Multi Agent (30/10)	0.800	0.767	0.653	0.810	0.628	0.732
Hybrid (30/10)	1.000 ★	0.867 ★	0.633	0.486	0.608	0.719
Hybrid Multi Agent (10/7)	0.800	0.703	0.805 ★	0.820	0.627	0.751
Hybrid Multi-Agent (15/10) ★	**0.800**	**0.733**	**0.753**	**0.826 ★**	**0.659**	**0.754 ★**
Hybrid Multi-Agent (20/10)	0.800	0.767	0.763	0.802	0.619	0.750
Hybrid Multi-Agent (30/10)	0.800	0.750	0.802	0.731	0.639	0.744

★ Best overall configuration

5. Analysis

The evaluation results reveal three distinct patterns that cut across all seven configurations. Each pattern has direct implications for the design of future RAG pipelines in specialised, high-precision domains like legal question answering.

5.1 Context Window Size is the Dominant Hyperparameter for Faithfulness

The most pronounced effect observed across the entire evaluation is the relationship between the number of documents fed to a sub-agent and that sub-agent's faithfulness score. Comparing Multi-Agent configurations at two different top_k settings reveals a faithfulness delta of 0.152 — the largest single hyperparameter effect in the study.

Configuration	Faithfulness	Answer Relevancy
Multi Agent (top_k=10, final=7)	0.805	0.820
Multi Agent (top_k=30, final=10)	0.653	0.810
Delta	+0.152	+0.010

The mechanism is straightforward: when a sub-agent receives 30 documents, a significant fraction of them will be marginally relevant or topically adjacent but not directly responsive to the query. The LLM, attempting to synthesize a comprehensive answer, draws on these peripheral documents and produces claims that are not grounded in the genuinely relevant subset — lowering the faithfulness score. With only 7–10 documents, the context is dense with on-point evidence, and the model has little room to hallucinate.

Answer relevancy, by contrast, is far less sensitive to this effect (delta of only 0.010), suggesting that relevancy is primarily governed by the quality of the final synthesis rather than the breadth of the input context.

This finding has a practical implication: for multi-agent architectures operating on legal corpora, the optimal top_k is not the largest feasible value but rather the smallest value that still achieves adequate recall. The Hybrid Multi-Agent results confirm this — faithfulness and relevancy both degrade monotonically as top_k increases from 15 to 20 to 30, while recall only marginally improves.

5.2 Retrieval Quality Alone Does Not Predict Answer Quality

The Hybrid RAG configuration (30/10) achieves the highest retrieval scores of all architectures — context precision of 1.000 means every single retrieved document was genuinely relevant, and context recall of 0.867 means nearly 87% of all relevant documents in the corpus were successfully retrieved. By standard information retrieval criteria, this is an excellent pipeline. Yet Hybrid RAG also produces the lowest answer relevancy score of the entire evaluation (0.486).

This apparent paradox is explained by the generation bottleneck: perfect retrieval is irrelevant if the generation step cannot process the retrieved evidence effectively. A single LLM call receiving 30 documents — even 30 perfectly relevant ones — faces a **context aggregation problem**. Legal documents from different jurisdictions, different procedural stages, and different articles of code may all be relevant in isolation but difficult to synthesize into a single coherent, focused answer. The model hedges, qualifies, and spans multiple sub-topics rather than identifying the single most responsive answer.

The Hybrid Multi-Agent architecture uses an identical retrieval pipeline — the same metadata extraction, two-phase FAISS, BM25+RRF, and cross-encoder steps — but routes the retrieved pool to sub-agents rather than a single LLM. Answer relevancy recovers from 0.486 to 0.826. This demonstrates conclusively that improving retrieval precision beyond a certain threshold yield diminishing returns unless the generation architecture is capable of consuming the evidence at the appropriate level of granularity.

5.3 Answer Correctness is Bounded by the Underlying Model

Answer correctness — the metric capturing factual overlap with ground-truth reference answers — shows the narrowest spread of all five metrics, ranging from 0.608 (Hybrid, 30/10) to 0.695 (Single Agent, 30/10), a range of just 0.087 points.

Strikingly, the simplest architecture achieves the highest correctness score. This counterintuitive result arises because the Single Agent passes all retrieved evidence directly to the LLM without any intermediate summarization step where details might be lost. Each additional layer of synthesis — sub-agent partial answers, supervisor consolidation — introduces a small probability of dropping or paraphrasing specific factual details present in the source documents.

More fundamentally, answer correctness measures alignment with a ground-truth reference answer, which is itself a fixed text. The LLM's capacity to reproduce the specific phrasing, citations, and factual claims in that reference is bounded by its parametric knowledge and its max_tokens budget. Increasing

max_tokens from 384 to 768 in the current configuration reduces truncation artefacts for long answers, but the ceiling on correctness is set by the model itself rather than by the retrieval or synthesis strategy. Meaningful further gains on this metric would require a larger or more legally specialised foundation model, or higher-quality (longer, more specific) ground-truth reference answers.

6. Conclusions

The evaluation demonstrates that the Hybrid Multi-Agent architecture at top_k=15, top_k_final=10 is the highest-performing configuration across the RAGAS framework, achieving a mean score of **0.754**. Its performance is the result of combining two complementary design choices :

— a hybrid retrieval pipeline that maximises the quality and completeness of the retrieved evidence pool,
— a multi-agent synthesis strategy that distributes that evidence across focused, isolated sub-agents rather than overwhelming a single LLM context window.

The following design principles emerge from this evaluation as reliable guidelines for building high-performance legal RAG systems:

- Compact sub-agent context (10–15 documents) consistently outperforms large context (30 documents) for both faithfulness and relevancy; the optimal top_k is the smallest value that achieves adequate recall, not the largest feasible value
- BM25+RRF hybrid retrieval meaningfully improves precision on legal text, where exact matches on article numbers, statute identifiers, and Latin phrases carry high discriminative value that dense embeddings alone may underweight
- Cross-encoder threshold filtering with a logit cutoff of 0.0 is an effective post-retrieval quality gate that removes marginally relevant documents before they can introduce noise into the generation step
- Structured LLM metadata extraction via LEGAL_METADATA_SCHEMA provides more reliable domain and jurisdiction routing than keyword heuristics, particularly for ambiguous or multi-domain queries
- Answer correctness is determined primarily by the underlying LLM's factual capacity and the quality of ground-truth references; it cannot be substantially improved by pipeline-level changes alone

Recommended next step:

- *Rebuild all four FAISS vector stores using BAAI/bge-base-en-v1.5 (768-dimensional embeddings) and re-run the full RAGAS evaluation.*
- *The higher-dimensional embedding space may capture more nuanced semantic relationships between legal concepts — particularly across jurisdictions where equivalent legal provisions use different terminology — and is expected to close the remaining gap in answer correctness that pipeline design changes alone cannot address.*